

Group Number: 27

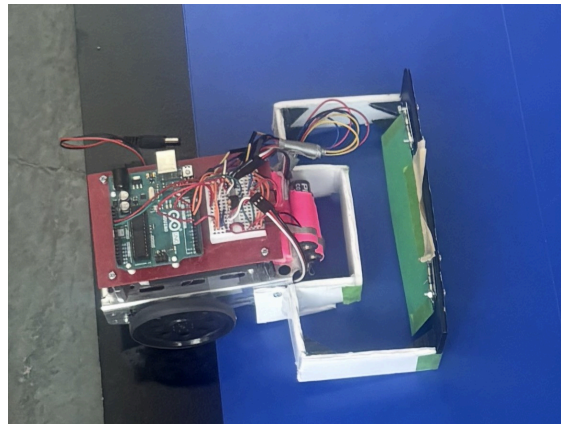
Teammates: Navya Penati (ngp42), Amanda Nicholson (agn39), Logan Roberts (llr63)

1. Robot Design and Strategy Overview

Our robot was designed to gather and retain as many cubes as possible within the one-minute competition period. The competition required the robot to begin within the 8-inch starting boundary, operate fully autonomously, and gather cubes and store them within the robot's perimeter when viewed from above.

Mechanically, our design centered around a collection and storage system inspired by a one-way gate. The original concept included three main components: a one-way fence that allowed cubes to enter but prevented them from leaving the robot, a ramp, and a storage space on top of the robot for the cubes. This design was intended to balance the offense and defense by allowing the robot to collect cubes while also reducing the chances for another robot to remove cubes from our possession. In our final design we decided to build a perimeter around the front half of our robot to store the cubes after they had gone through our one-sided gate.

Electrically the robot used motors controlled-through an H-bridge for forward, reverse, and turning movement. The design also included sensors to help the robot detect its environment, including two QTI sensors for detecting the blue and yellow sides of the field and for border detection. The software strategy was to make the robot fully autonomous by using sensor input to determine when the robot should move forward, backward, turn, avoid the black border, and continue searching for and collecting cubes.



2. Design Process Reflection

Our group began the project by identifying the main design constraints and competition requirements. The robot had to fit within the 8-inch starting boundary, expand or operate within the allowed competition-size, gather cubes, avoid losing cubes, remain fully autonomous, and stay within the project material and budget

constraints. Based on these constraints, we decided to prioritize a cube collection system that could collect cubes and retain them.

The first design concept included a ramp, one-way fence, and an upper storage area. The goal was for cubes to travel up the ramp and remain stored on the robot (Figure 2). The goal was for the cubes to travel up the ramp and remain stored on the robot. As we progressed through the milestones, we tested the robot's basic mobility first, including forward motion, backward motion, and left and right turns. This was necessary because the milestone requirements expected the robot to complete a path with these movements.

After achieving basic mobility we focused on sensor behavior. One of the biggest challenges we faced was getting the QTI sensors to work consistently. At first, the robot was not able to tell the difference between the blue and yellow sides of the board. After testing, we realized that the QTI sensors had to be extremely close to the board in order to read the colors accurately, so we adjusted their placement to keep them closer to the surface of the board. Later, once we added the cardboard border around the robot, we found that the border was interfering with the QTI sensors and preventing them from reading correctly. To fix this, we had to cut part of the cardboard border off so the sensors could detect the colors properly.

This sensor testing was important because the robot needed to identify the board color, move toward the opposite color, turn around, and stay on the board. The QTI sensors forced us to adjust both the robot's physical design and the code.

A major challenge was balancing our mechanical design with the time needed for coding and testing. As a result, we simplified parts of the robot so it could perform more reliably on competition day. We took off parts such as the ramp and storage space because it would cause our robot to move slower. Overall, our design process moved from ambitious collection mechanisms toward a more practical robot that could satisfy the milestones and compete effectively.

3. Competition Analysis

Overall, our robot performed fairly well during the competition. At first, the robot was not moving forward and was only spinning in circles. After troubleshooting, we realized that the issue was related to the QTI sensors, so we unplugged them and simplified the code. Instead of relying on sensor input, the robot was programmed to drive forward, turn right near the line, and continue moving through a time sequence. This strategy ended up being useful because the robot was able to collect many cubes. The main strength of our robot was that its structure was very sturdy and that once the program was simplified, it moved more consistently and was able to collect and store cubes.

The biggest weakness was that the robot sometimes drove off the board. Since the QTI sensors were unplugged, the robot could no longer detect the black border. We

realized that the robot was programmed to stop too late, which caused it to run off the edge. Because of this, when the robot ran off of the edge of the board it lost a lot of the cubes it had stored. To fix this during the later competitions, we angled the robot's starting position so it would stay on the board for most of its program.

The competition showed that our robot worked best with a simple and reliable movement strategy. However, better QTI sensor reliability would have helped the robot detect the border and avoid leaving the board while still collecting cubes.

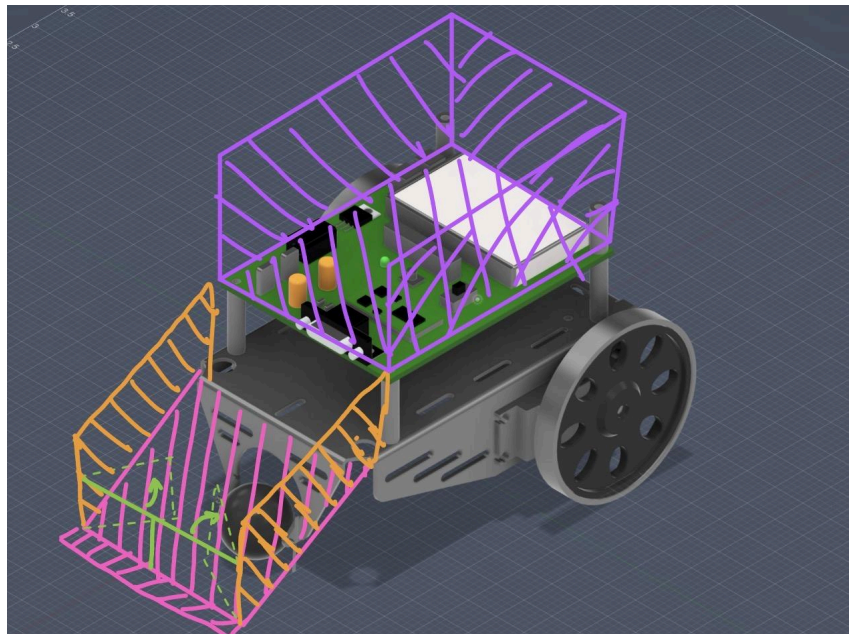


Figure 2: Sketch of Robot

4. Conclusions

Overall, this project showed the importance of keeping the robot design simple, reliable, and easy to troubleshoot. Our robot was able to collect cubes effectively once we simplified the program, but the QTI sensor issues made it harder to consistently detect the black border and stay on the board.

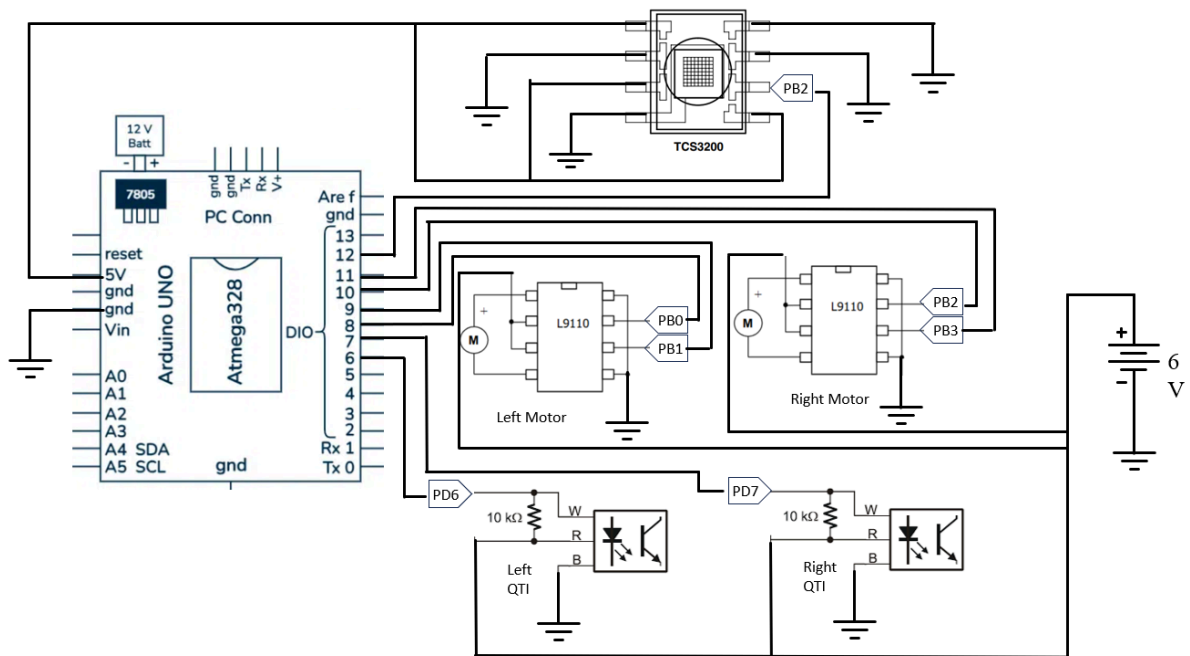
If we could do this project again, we would spend more time testing the sensors earlier on and making sure their placement was reliable before adding the full cardboard border. We would have also done lots of testing before the final competition to ensure our the final product of our robot worked. We would also make the robot's movement depend more on the sensor's feedback instead of timed movements, since the hard-coded timing caused the robot to sometimes drive off the board.

Our advice to students doing this project next year would be to test early, keep the design simple, and make sure each part of the robot works before adding more complexity.

Appendix A - Bill of Materials

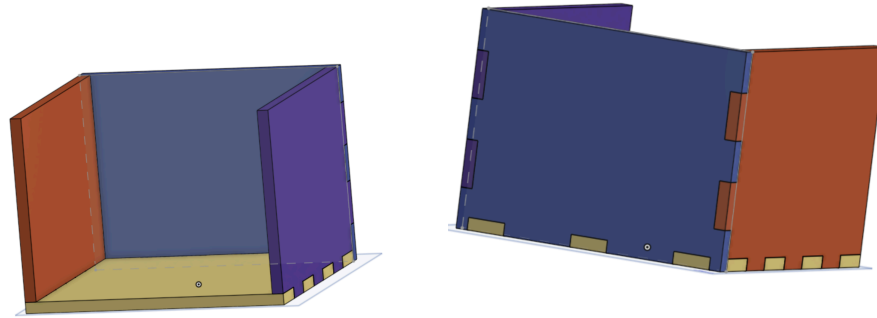
Bill of Materials			
Amount	Item	Cost	Use
2	Positional Servo	2 x (\$3)	Unused
1	¼ Inch, 12 x 12 Acrylic Sheet	\$10.83	Unused
1	Metal Scrap Sheet	\$5	Unused
1	20 x 30 Corrugated Plastic Board	\$4.99	Used
1	20 x 30 Foam Board	\$4.99	Used
2	Spring Hinge	2 x (\$1.66)	Used
TOTAL: \$35.13			

Appendix B - Circuit Diagram

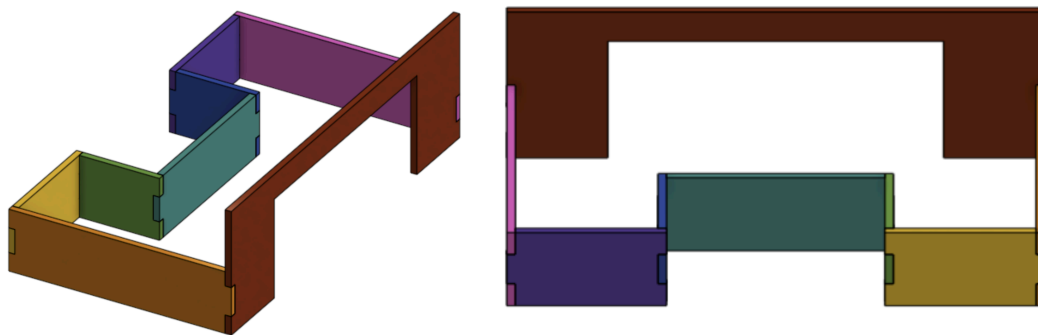


Appendix C - CAD Diagrams

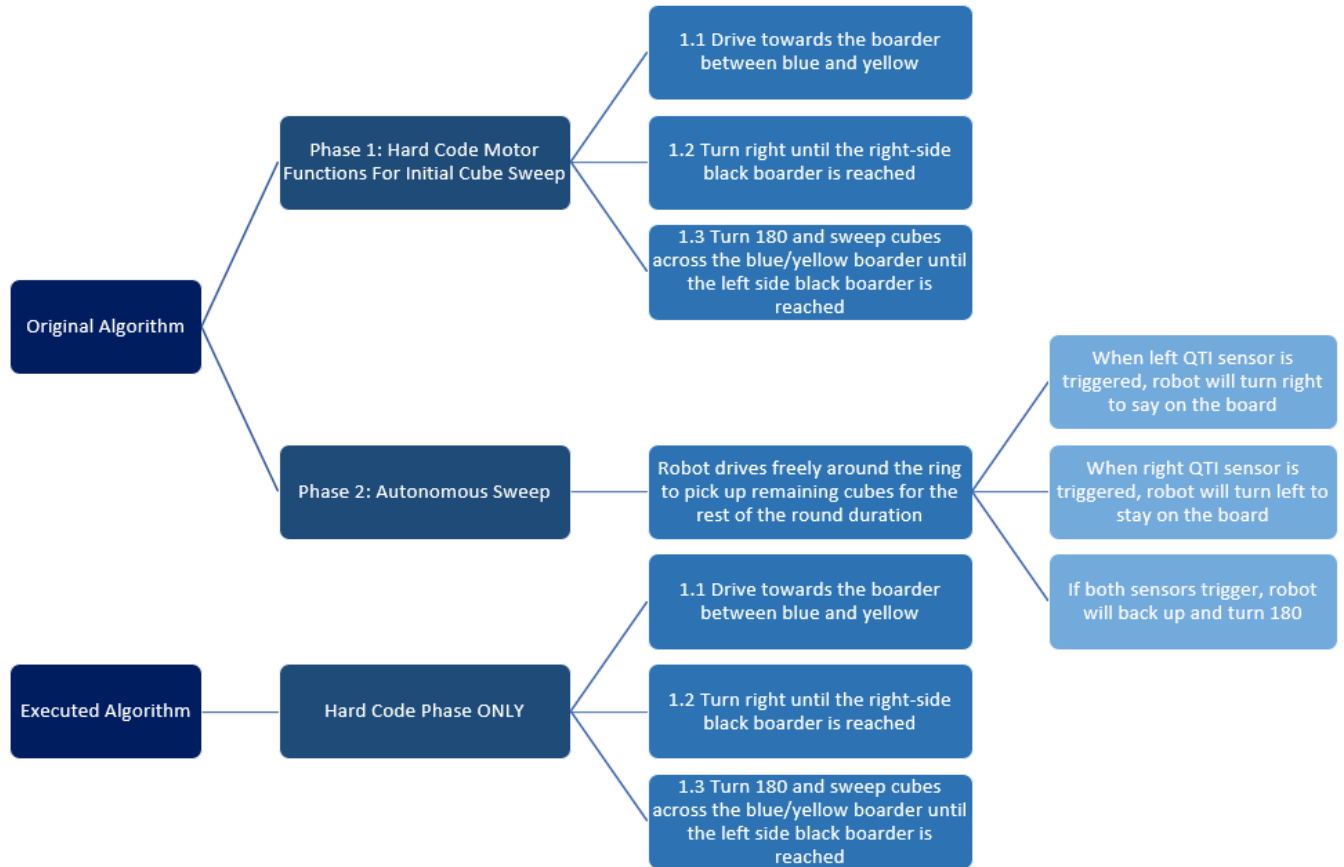
This CAD model shows our original design. The cubes were meant to be pushed up a ramp and collected in the box. However, the box ended up being too large for our robot, and the overall design was too complicated, so we decided not to use it and changed our design.



This CAD model shows the design we used for our final robot. It functions like a gate. The cubes enter through the gate and are corralled into a pen. A paper flap is taped to the inside of the gate to prevent the cubes from escaping. We ended up not laser cutting this design. Instead, we used the CAD model as a guide to make the parts out of posterboard.



Appendix D - Flow Chart



Appendix E - Code

```

#include <avr/io.h>
#include <util/delay.h>
#include <avr/interrupt.h>
#include <Arduino.h>

// global variables
//volatile int period;
//volatile int timer;

float distance = 0;
volatile float pulse_duration = 0;
volatile int input_flag = 1;
const float v_sound = 340;
  
```

```

// motor functions
void drive_forward(void);
void drive_backward(void);
void turn_left(void);
void turn_right(void);
void stop_motors(void);

// Right QTI
int getQTIRight(void){
    if(PIND & 0b10000000){
        return 1;
    }
    else{
        return 0;
    }
}

// Left QTI
int getQTILeft(void){
    if(PIND & 0b01000000){
        return 1;
    }
    else{
        return 0;
    }
}

// ISR interrupt on D5
ISR(PCINT2_vect){
    if (PIND & 0b00100000){ // if pin 5 is high
        TCNT1 = 0; // timer to 0
    }else{ // pin 5 is low, falling edge of input pulse
        pulse_duration = TCNT1;
    }
}

```

```

    input_flag = 0;
}
}

void initUltrasonic(void){

    PCICR = 0b00000100;
    PCMSK2 = 0b00100000; // enable PCINT23 on pin 5
    TCCR1A = 0b00000000; // normal mode
    TCCR1B = 0b00000010; // timer1 prescaler to 8
}

float readUltrasonic(void){
    input_flag = 1;

    DDRD |= 0b00100000; // D5 output
    PORTD |= 0b00100000; // trigger high

    _delay_us(5); // 5 microsecond pulse

    PORTD &= 0b11011111; // trigger low
    DDRD &= 0b11011111; // D5 input

    TCNT1 = 0;

    while(input_flag == 1 && TCNT1 < 60000){

    }

    if(input_flag == 1){
        return 999; // no echo detected
    }

    return v_sound * pulse_duration * 8 * 62.5e-9 * 100 / 2;
}

```

```
}

// main
int main(void){

    Serial.begin(9600); // initialize serial
    //initColor();
    //getColor();

    DDRB |= 0b00001111; //motor output

    initUltrasonic();
    sei();

    volatile int valRight;
    volatile int valLeft;
    int mode = 0;
    float obstacleDistance = 15.0;

    drive_forward();
    _delay_ms(3000);

    turn_right();
    _delay_ms(570);

    drive_forward();
    _delay_ms(5000);

    turn_right();
    _delay_ms(1150);

    turn_right();
    _delay_ms(570);
```

```
    drive_forward();  
    _delay_ms(1000);  
  
    stop_motors();  
  
}  
  
// ----- MOTOR LOGIC -----  
void drive_forward(void){  
    PORTB = (PORTB & 0b11110000) | 0b00000101;  
}  
  
void drive_backward(void){  
    PORTB = (PORTB & 0b11110000) | 0b00001010;  
}  
  
void turn_left(void){  
    PORTB = (PORTB & 0b11110000) | 0b00000110;  
}  
  
void turn_right(void){  
    PORTB = (PORTB & 0b11110000) | 0b00001001;  
}  
  
void stop_motors(void){  
    PORTB = (PORTB & 0b11110000);  
}
```